# Remaining Useful Life Prediction for Turbofan Engines: Sequence Models and a Hybrid Feature-Fusion Network

Kushal Tyagi
Enrollment No. 2401010249

Nishant Sharma
Enrollment No. 2401010302

Anant Lalwani
Enrollment No. 2401020086

Dev Tyagi
Enrollment No. 2401010148

Vansh Khod
Enrollment No. 2401010493

Newton School of Technology, Rishihood University
Advanced Machine Learning · Project 7, Phase 2 (Deep Learning)

Abstract—We compare models that learn remaining useful life (RUL) directly from sensor sequences with the engineered-feature regression of Phase 1 on NASA's C-MAPSS FD001 benchmark. A leakage-free pipeline turns every cycle into a 30-cycle window of 14 normalised sensors, keeps short histories through padding with a mask channel, and reuses Phase 1's engine-level folds. An LSTM, a temporal convolutional network and a Transformer encoder all learn real degradation patterns (test RMSE 14.7–15.5, against 18.2 for XGBoost on raw single-cycle sensors), but are significantly worse than XGBoost on 88 engineered features (10.75; paired Wilcoxon $p \leq 0.0015$). Experiments trace the gap to information outside any fixed window: each engine's distance from its own healthy baseline, and its age. A hybrid network that fuses an LSTM encoder with the engineered features recovers the gap. It beats both of its branches and late fusion in cross-validation, is statistically level with XGBoost on the official benchmark (RMSE 11.02, $p = 0.90$), and has the lowest error over all test cycles in every degradation stage. Ablations show that the features are the backbone (+4.5 RMSE without them) and that longer windows help sequence-only models but not the hybrid. Retraining with different seeds moves a CV score by about ±0.5–0.7, so smaller design effects cannot be distinguished from training noise. Gradient saliency shows that the sequence branch relies on the core-speed sensors that the engineered features use least.

Index Terms—remaining useful life, C-MAPSS, LSTM, temporal convolutional network, Transformer, hybrid model, ablation study

## I. Introduction

Remaining useful life (RUL) prediction estimates how many operating cycles an engine has left before it fails, from its multivariate sensor history [1]. Project 7 compares two routes on NASA's C-MAPSS FD001 benchmark [2], [3]: engineered degradation features with classical regression, and models that learn directly from sensor sequences.

Phase 1 built the first route. A leakage-free pipeline of 88 causal features (rolling statistics, trend slopes, deviation from each engine's own early-life baseline and a PCA health index) with XGBoost [4] reached a test RMSE of 10.75 cycles. Its error analysis also identified where fixed-window features fall short: fast-degrading engines, the onset of degradation, and short histories. All three depend on the shape and timing of the full trajectory.

Phase 2 tests the second route and a combination of both. Our contributions are:

- a sequence-data pipeline that keeps every cycle (including short histories, via padding and a mask channel) and is provably free of cross-engine and future leakage;
- an evaluation of the three architectures named in the brief, LSTM, temporal CNN and Transformer encoder, on the same engine-level folds as Phase 1;
- a hybrid feature-fusion network that combines a sequence encoder with the Phase 1 degradation features, justified by a specific blind spot of window-based models;
- ablation studies of branches, window length, regularisation and the padding mask;
- a validation on the official test set with three seeds, bootstrap confidence intervals, a paired significance test against Phase 1, stage-wise errors and gradient saliency.

## II. Related Work: Deep Learning for RUL

Convolutional models. Babu et al. [5] first applied a CNN to C-MAPSS sensor windows (FD001 RMSE 18.45). Li et al. [6] used a deeper 1-D CNN over 30-cycle windows with the piecewise-linear target and reported 12.61. Bai et al. [7] showed more generally that temporal convolutional networks (TCN) with causal, dilated convolutions, as introduced in WaveNet [8], and residual connections [9] match or beat recurrent networks on many sequence tasks while training in parallel.

Recurrent models. The LSTM [10] adds a gated, additively updated cell state to the recurrent network, so gradients survive over long sequences. Zheng et al. [11] applied stacked LSTMs to C-MAPSS (FD001 RMSE 16.14),

and Heimes [12] had already used recurrent networks in the PHM'08 challenge.

Attention. The Transformer [13] replaces recurrence with self-attention, relating every pair of time steps directly. Mo et al. [14] combined a Transformer encoder with a gated convolutional unit for RUL estimation on C-MAPSS.

Hybrid models. Al-Dulaimi et al. [15] ran LSTM and CNN branches in parallel and fused them, and Chen et al. [16] combined attention-based LSTM features with handcrafted features. Both support the idea of fusing complementary representations; our hybrid follows the second line, fusing a learned sequence representation with the engineered features of Phase 1. As in Phase 1, published numbers are indicative only, because preprocessing and evaluation protocols differ [17].

## III. Sequence Data and Regularisation

### A. Windows within engine boundaries

For engine $i$ at cycle $t$, the model input is the window of the last $W$ cycles,

$$X_{i,t} = [z_{i,t-W+1}, \ldots, z_{i,t}] \in \mathbb{R}^{W \times 14}, \tag{1}$$

where $z$ are the 14 informative sensors after the same z-score normalisation as Phase 1, fitted on training engines only. With stride one, every row of the data becomes one window (20,631 training windows), so both phases are trained and scored on exactly the same cycles. The target is the Phase 1 piecewise-linear RUL capped at 125 [12], divided by 125 so that it lies in $[0, 1]$.

### B. Short histories: padding and mask

Cycles with fewer than $W$ predecessors are kept rather than discarded: the window is left-padded with the engine's own first reading, and a fifteenth channel marks real (1) and padded (0) steps. Without the mask, a padded window would look like a perfectly flat, healthy engine. With $W = 30$, 14% of training cycles and 22% of test cycles have partly padded windows; at $W = 80$ this rises to 38% and 57%, and 23 test engines are shorter than the window.

### C. Leakage control

Three properties are checked in the notebook and in automated tests: (i) every window contains only its own engine's cycles (rebuilding each engine's windows in isolation gives identical arrays); (ii) windows are causal (deleting all future cycles leaves every remaining window unchanged, maximum difference 0); (iii) normalisation statistics and the engineered features of the hybrid are refitted inside every fold.

### D. Splits

We reuse Phase 1's five folds over engine ids (20 evaluation engines per fold), so CV scores are directly comparable across phases. Inside each fold, 10% of the training engines are held out as an inner validation set for early stopping and the learning-rate schedule; the evaluation engines are never used for any training decision. The official test set is used once.

### E. Regularisation

FD001 has only 100 training engines and consecutive windows overlap in $W - 1$ cycles, so networks with tens of thousands of weights overfit quickly: in a demonstration run the inner-validation RMSE was lowest at epoch 10 and rose afterwards while the training error kept falling. We therefore combine dropout ($p = 0.2$) [18]; decoupled weight decay ($\lambda = 10^{-4}$) with the AdamW optimiser [19], [20], i.e. minimising

$$\mathcal{L}(w) = \frac{1}{n} \sum_{j} \left(\hat{y}_j(w) - y_j\right)^2 + \lambda \|w\|_2^2; \tag{2}$$

early stopping with patience 8 and restoration of the best weights [21]; halving the learning rate on validation plateaus; and gradient-norm clipping at 1.0, which protects recurrent networks from exploding gradients [22]. Gaussian input noise ($\sigma = 0.05$) is evaluated as an augmentation in the ablations.

## IV. Sequence Architectures

All three networks map a window $X \in \mathbb{R}^{30 \times 15}$ to an embedding $h$, followed by the same head (32 ReLU units, dropout, one linear output).

LSTM. Two stacked layers of 64 units read the window cycle by cycle:

$$\begin{aligned} &f_t = \sigma(W_f[h_{t-1}, x_t] + b_f), \quad i_t = \sigma(W_i[h_{t-1}, x_t] + b_i), \\ &c_t = f_t \odot c_{t-1} + i_t \odot \tanh(W_c[h_{t-1}, x_t] + b_c), \\ &h_t = \sigma(W_o[h_{t-1}, x_t] + b_o) \odot \tanh(c_t). \end{aligned} \tag{3}$$

The forget and input gates decide what the cell state keeps and adds; because $c_t$ is updated additively, the gradient along it does not shrink geometrically with the sequence length. The final hidden state is the embedding (56k parameters).

TCN. Residual blocks of two causal 1-D convolutions (kernel $k = 3$, 64 channels) with dilations $d_l = 1, 2, 4, 8$ give a receptive field

$$R = 1 + \sum_{l} 2(k-1)\, d_l = 61 \text{ cycles} \geq W. \tag{4}$$

Causality (left padding only) guarantees that the output at cycle $t$ uses no later input; the embedding is the output at the last cycle (93k parameters).

Transformer encoder. Inputs are projected to width 64, sinusoidal positional encodings are added, and two pre-norm encoder layers with four heads apply

$$\text{Attention}(Q, K, V) = \text{softmax}\Big(QK^\top / \sqrt{d_k}\Big)\, V, \tag{5}$$

TABLE I
5-fold CV RMSE (capped target, same folds as Phase 1)

| Model (input) | RMSE (mean ± std) | MAE |
|---|---|---|
| XGBoost (raw, one cycle) | 18.57 ± 0.53 | 13.49 |
| TCN (window) | 15.41 ± 0.95 | 11.65 |
| LSTM (window) | 14.01 ± 1.23 | 10.20 |
| Transformer (window) | 13.05 ± 0.73 | 8.74 |
| XGBoost (88 features) | 9.26 ± 0.50 | 6.30 |

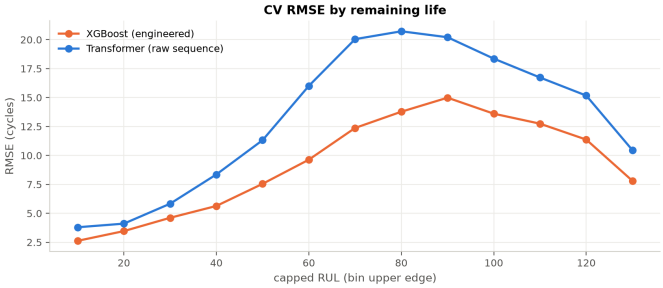



Fig. 1. CV RMSE by remaining life: the sequence model's gap to XGBoost is largest at the onset of degradation.

with padded steps removed from attention by the mask. The output at the current cycle is the embedding (70k parameters).

### A. Cross-validation results

Table I compares the three networks with the Phase 1 models on the same folds. All networks beat XGBoost on single-cycle raw sensors (18.57) by 3 to 5.5 cycles, so learning from the 30-cycle history is clearly useful. The Transformer is best ($13.05 \pm 0.73$), followed by the LSTM and the TCN. None reaches the engineered-feature XGBoost (9.26). Their errors are largest in the onset region (RUL 60–100, up to 20 cycles for the Transformer against 13–15 for XGBoost), where the drift within 30 cycles is small compared with the noise. The likely reason is that a 30-cycle window cannot see the engine's own healthy baseline or its age, the two most important pieces of information in Phase 1, and 100 engines are too few for a network to learn such long-range references from scratch.

## V. Hybrid Feature-Fusion Model

### A. Rationale

The two routes see an engine differently, and each misses what the other has. The engineered features summarise information that lies outside any fixed window, most importantly the distance of each sensor from the engine's own first ten cycles, which Phase 1 found to be the most informative feature group, and the engine's age. A window-based network cannot compute these, because the first cycles of a long-lived engine are not in its 30-cycle window. Conversely, the features compress the recent history into a few hand-chosen statistics and cannot

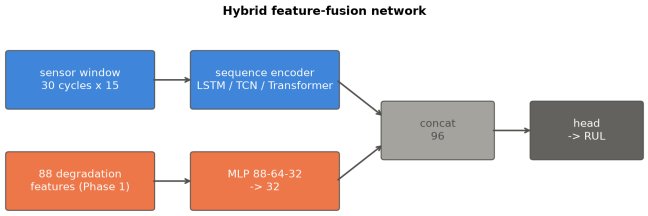



Fig. 2. Hybrid feature-fusion network.

represent trajectory shapes that the designer did not anticipate. Section IV showed the cost of the first gap: the networks fall 4–6 cycles behind XGBoost, most of all at the onset of degradation. A hybrid that gives the network both views addresses this specific blind spot, which is the "degradation feature engineering → sequence-based prediction" logic of the project brief.

### B. Architecture

$$h_{\text{seq}} = \text{Enc}(X_{i,t}), \qquad h_{\text{feat}} = \text{MLP}(f_{i,t}),$$
$$\hat{y}_{i,t} = g([h_{\text{seq}}; h_{\text{feat}}]), \tag{6}$$

where Enc is one of the three encoders of Section IV (64-d output), $f_{i,t}$ are the 88 standardised Phase 1 features at cycle $t$ (causal, refitted per fold), the MLP is $88 \to 64 \to 32$ with ReLU and dropout, and $g$ is $96 \to 64 \to 1$. Both branches are trained jointly with one loss, so the encoder is pushed to learn what the features do not already explain. This early (feature-level) fusion differs from averaging two separately trained models (late fusion), which we test as a baseline.

### C. Results

Table II reports the hybrid against its two branches, Phase 1's XGBoost, and the late-fusion average. With the LSTM encoder, the hybrid ($9.55 \pm 0.77$) is 4.5 cycles better than the best sequence model alone and 0.5 cycles better than the feature branch alone, so each branch contributes information. Joint training matters: averaging XGBoost and the Transformer scores 10.24, worse than XGBoost alone, while the hybrid beats the average with every encoder. Against XGBoost (9.26) the hybrid is close but not better: it wins on 2 of 5 folds, and averaged over three seeds (Section VI) its CV RMSE is about 9.7–10.0.

The gain is concentrated where it matters most. In the last 50 cycles before failure the hybrid's RMSE is 4.14 against 5.04 for XGBoost (18% lower) and 7.20 for the Transformer alone. In the onset region it recovers almost all of the sequence model's weakness (14.31 vs. 19.07; XGBoost 13.80). Fast-degrading engines remain about 10 cycles late for both XGBoost and the hybrid, since both use the engine's age, which makes a fast degrader look too young to fail.

TABLE II
Hybrid versus its parts (5-fold CV RMSE)

| Model | RMSE | RMSE, last 50 cycles |
|---|---|---|
| Sequence alone (Transformer) | 13.05 | 7.20 |
| Feature branch alone (MLP) | 10.04 | – |
| Average of XGBoost and Transformer | 10.24 | – |
| XGBoost, engineered (Phase 1) | 9.26 | 5.04 |
| Hybrid, TCN encoder | 9.87 | – |
| Hybrid, Transformer encoder | 9.74 | – |
| Hybrid, LSTM encoder | 9.55 | 4.14 |

TABLE III
Ablations of the LSTM hybrid (5-fold CV RMSE, one seed)

| Variant | RMSE | $\Delta$ vs. hybrid |
|---|---|---|
| Hybrid ($W = 30$, early stopping) | 9.55 | – |
| Without sequence branch | 10.04 | +0.49 |
| Without feature branch | 14.01 | +4.46 |
| $W = 50$ / $W = 80$ | 9.49 / 9.51 | −0.06 / −0.04 |
| Without dropout | 9.67 | +0.12 |
| Without weight decay | 9.52 | −0.03 |
| Fixed 60 epochs (no early stopping) | 9.20 | −0.36 |
| With input noise ($\sigma = 0.05$) | 9.60 | +0.05 |
| Without padding mask | 9.74 | +0.18 |

TABLE IV
Seed variability of the hybrid's CV RMSE

| Configuration | seed 42 | seed 1 | seed 2 | mean ± std |
|---|---|---|---|---|
| Early stopping (default) | 9.55 | 10.81 | 9.61 | $9.99 \pm 0.71$ |
| Fixed 60 epochs (final) | 9.20 | 10.27 | 9.71 | $\mathbf{9.73 \pm 0.54}$ |

## VI. Ablation Studies

Each ablation changes one component of the LSTM hybrid and keeps everything else fixed (folds, inner validation engines, seed). Table III and Fig. 3 report the change in 5-fold CV RMSE. Before interpreting them, we measured how much a CV score moves when the same configuration is retrained with different seeds (Table IV): about ±0.5–0.7 cycles. Re-running one ablation with an identical configuration also gave 9.39 and then 9.60, because parallel floating-point arithmetic on a multi-core CPU is not bit-reproducible. We therefore treat only effects clearly larger than about 0.5 cycles as established.

Branches. Removing the feature branch costs +4.46 cycles (worse on all five folds), far beyond the noise: the engineered features are the backbone. Removing the sequence branch costs +0.49 (worse on four of five folds). That is consistent across folds but of the same size as seed noise, so the value of the sequence branch rests on several pieces of evidence together: this ablation, the stage-wise test results (Section VII) and the saliency analysis.

Window length. This ablation tests the Phase 1 hypothesis that fixed windows miss long degradation trajectories, and it is the clearest result. For the LSTM alone, a longer history helps markedly: RMSE falls from 14.01 ($W = 30$) to 11.87 ($W = 50$), then rises to 12.46 at $W = 80$, where 38% of training cycles become partly padded. The hybrid barely changes (9.55, 9.49, 9.51), because its engineered features already carry the long-range context. We keep $W = 30$, which gives the same accuracy at a fraction of the cost (3 versus 44 minutes of CV training at $W = 80$).

Regularisation and mask. Dropout (+0.12 without it), weight decay (−0.03), Gaussian input noise (+0.05) and the padding mask (+0.18 without it) all change the error by less than the seed noise, so none of them can be shown to matter on its own at this data size. Early stopping is different. Its signal comes from only 7–8 inner-validation engines, and it stopped at epochs 14–30. Trained for a fixed 60 epochs, the hybrid's validation error ends within 0.05–0.4 cycles of its best value on every fold, so dropout and the learning-rate schedule are enough to prevent overfitting. For the sequence-only networks, whose validation error turned up after 7–16 epochs, early stopping remains necessary.

Final configuration. A single run with a fixed 60 epochs gave the lowest CV RMSE (9.20). Over three seeds, it remains better and steadier than the default ($9.73 \pm 0.54$ vs. $9.99 \pm 0.71$), and it was adopted for the test evaluation. The choice used only the CV folds. Averaged over seeds, the hybrid's CV RMSE is slightly behind XGBoost (9.26).

## VII. Test-Set Validation

The models were retrained on all 100 training engines (10 held out for validation) with three seeds each, and evaluated once on the official test set, with the same protocol as Phase 1. The reported prediction of each network is the average of its three seeds. Phase 1's tuned XGBoost was retrained alongside so that all models are scored on identical rows.

### A. Official benchmark

Table V reports the last-cycle results against the true RUL with 95% bootstrap intervals over engines. The hybrid scores 11.02 (9.3–12.7), level with XGBoost (10.75, 9.1–12.4), with a slightly lower MAE (8.02 vs. 8.09). Its three seeds score 11.05–11.68, so seed variation is of the same size as the differences between the best models. The sequence-only networks score 14.7–15.5: better than XGBoost on raw single-cycle sensors (18.19), but clearly behind the engineered features, the same ranking as in cross-validation.

### B. Significance

Because all models are scored on the same engines, we compare them with a paired Wilcoxon signed-rank test [23] on per-engine absolute errors, which does not assume normally distributed errors. Each sequence-only network is significantly worse than XGBoost ($p \leq 0.0015$; better on only 33–40 of 100 engines). The hybrid and XGBoost are statistically indistinguishable ($p = 0.90$; the hybrid is better on 51 engines). Their last-cycle errors are strongly correlated ($r = 0.86$): the same engines are hard for both.

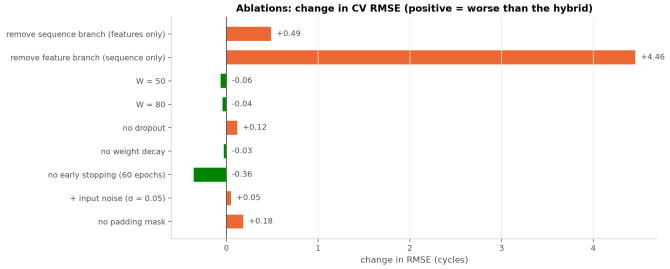



Fig. 3. Change in CV RMSE for each ablation (positive = worse). Changes below about 0.5 are within seed noise.

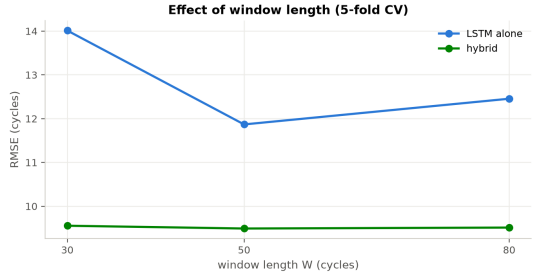



Fig. 4. Window length: the sequence-only LSTM gains from longer histories; the hybrid does not need them.

### C. Error by degradation stage

Scoring every test cycle (13,096 points) against the capped target, the hybrid has a lower RMSE than XGBoost in every stage: early 5.95 vs. 6.44, mid 12.33 vs. 12.44, late 5.37 vs. 5.70, and 8.54 vs. 8.81 overall (Fig. 5). Its late-stage bias stays small (+2.9 cycles), while the LSTM alone is biased late by +6.3 cycles near failure, the dangerous direction. At the last cycle of the 33 engines closest to failure (true RUL $\leq$ 50), the hybrid has the lowest MAE (2.93, vs. 3.38 for XGBoost and 4.39 for the LSTM). Fast-degrading engines remain about 10 cycles late for every model.

### D. What the sequence branch looks at

Gradient saliency [24], the mean $|\partial\hat{y}/\partial x_{s,\tau}|$ over test windows with RUL $\leq$ 80 (Fig. 6), shows that the network concentrates on the last ten or so cycles and on the core speeds $s_9$ (Nc) and $s_{14}$ (NRc), the HPC outlet pressure $s_7$ and the temperature $s_4$. The core speeds were the weakest sensors in Phase 1's ranking and carried little weight in the PCA health index. The sequence branch therefore exploits exactly the signal that the hand-made features use least, which is direct evidence of the complementarity behind the hybrid.

## VIII. Discussion and Limitations

The central comparison of the project has a clear answer on FD001: models that learn only from sensor sequences learn real degradation patterns (3–3.5 cycles better than raw single-cycle features) but do not reach carefully engineered degradation features. The reason is identified by the experiments: the most informative quantities, each engine's distance from its own healthy baseline and its age, lie outside any fixed window, and 100 engines are too few to learn such references. Longer windows help a sequence-only model (14.0 to 11.9 at $W = 50$), which supports the brief's hypothesis that fixed windows miss long trajectories. Supplying the context through engineered features helps more.

The hybrid recovers the whole gap, beats both its parts and late fusion in cross-validation, and matches the best feature-based model on the official benchmark, with a small but consistent advantage over all test cycles and near failure. We do not claim a significant improvement over XGBoost: the paired test shows none.

Limitations. FD001 has one operating condition, one fault mode and 100 test engines, so confidence intervals are about $\pm$1.7 cycles wide. Network training is noisy: retraining with another seed moves a CV score by about $\pm$0.5–0.7 cycles, as large as most design effects we measured, so only larger effects are claimed. Hyperparameters of the networks were not searched as extensively as XGBoost's in Phase 1, partly for this reason, and early stopping relied on only 7–8 inner-validation engines. Saliency is a local sensitivity measure, not a causal explanation.

Future work. Fast-degrading engines are predicted late by every model, because all of them use the engine's age. Features or targets that express degradation rate rather than age, and evaluation on the multi-condition subsets FD002–FD004 (our regime-normalisation pipeline already supports them), are the natural next steps.

TABLE V
Official FD001 test set (last cycle, true RUL)

| Model | RMSE [95% CI] | MAE | Score |
|---|---|---|---|
| XGBoost, raw sensors (Ph. 1) | 18.19 [14.9, 21.2] | 12.93 | 963 |
| LSTM, sensor windows | 15.52 [13.0, 18.0] | 11.63 | 463 |
| Transformer, sensor windows | 15.03 [12.7, 17.3] | 11.17 | 367 |
| TCN, sensor windows | 14.72 [12.6, 16.7] | 11.21 | 322 |
| XGBoost, engineered (Ph. 1) | 10.75 [9.1, 12.4] | 8.09 | 161 |
| Hybrid (LSTM + features) | 11.02 [9.3, 12.7] | 8.02 | 166 |

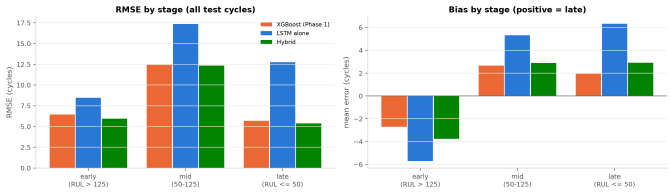



Fig. 5. Test RMSE and bias by degradation stage over all test cycles.

## IX. Conclusion

We built a leakage-free sequence-learning pipeline for RUL prediction and compared an LSTM, a temporal CNN and a Transformer encoder with Phase 1's engineered-feature regression under identical engine-level protocols.

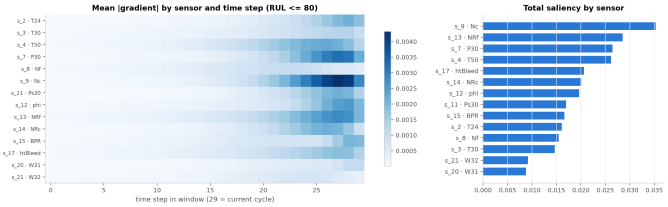



Fig. 6. Gradient saliency of the hybrid's sequence branch by sensor and time step.

On their own, the sequence models are significantly worse than the engineered features (test RMSE 14.7–15.5 vs. 10.75). A hybrid that fuses an LSTM encoder with the engineered degradation features closes this gap (11.02, statistically level with XGBoost), has the lowest error over all test cycles in every degradation stage, and its sequence branch draws on sensors the features under-use. On 100 engines, domain-informed degradation features remain the backbone of accurate RUL prediction, and sequence learning is most valuable as a complement to them rather than a replacement.

Team Contributions

| Member | Phase 2 responsibility |
|---|---|
| Kushal Tyagi | Sequence data pipeline, leakage checks, training loop and regularisation (notebook 06, src/sequences.py, src/training.py) |
| Anant Lalwani | Deep learning literature; LSTM, TCN and Transformer architectures and their comparison (notebook 07, src/dl_models.py) |
| Nishant Sharma | Hybrid feature-fusion model: rationale, design and evaluation (notebook 08, src/hybrid.py) |
| Dev Tyagi | Ablation studies and final configuration (notebook 09) |
| Vansh Khod | Test-set validation: seeds, bootstrap, significance, stages, saliency (notebook 10, src/validation.py) |

Code and notebooks: https://github.com/WhiteHacker000/Remaining-Useful-Life-Prediction-for-Industrial-Engines

References


[1] Y. Lei, N. Li, L. Guo, N. Li, T. Yan, and J. Lin, "Machinery health prognostics: A systematic review from data acquisition to RUL prediction," Mechanical Systems and Signal Processing, vol. 104, pp. 799–834, 2018.

[2] A. Saxena, K. Goebel, D. Simon, and N. Eklund, "Damage propagation modeling for aircraft engine run-to-failure simulation," in Proc. Int. Conf. Prognostics and Health Management (PHM), Denver, CO, USA, 2008, pp. 1–9.

[3] A. Saxena and K. Goebel, "Turbofan engine degradation simulation data set," NASA Ames Prognostics Data Repository, NASA Ames Research Center, Moffett Field, CA, 2008.

[4] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, 2016, pp. 785–794.

[5] G. Sateesh Babu, P. Zhao, and X.-L. Li, "Deep convolutional neural network based regression approach for estimation of remaining useful life," in Database Systems for Advanced Applications (DASFAA), ser. Lecture Notes in Computer Science, vol. 9642. Springer, 2016, pp. 214–228.

[6] X. Li, Q. Ding, and J.-Q. Sun, "Remaining useful life estimation in prognostics using deep convolution neural networks," Reliability Engineering & System Safety, vol. 172, pp. 1–11, 2018.

[7] S. Bai, J. Z. Kolter, and V. Koltun, "An empirical evaluation of generic convolutional and recurrent networks for sequence modeling," arXiv preprint arXiv:1803.01271, 2018.

[8] A. van den Oord, S. Dieleman, H. Zen, K. Simonyan, O. Vinyals, A. Graves, N. Kalchbrenner, A. Senior, and K. Kavukcuoglu, "WaveNet: A generative model for raw audio," arXiv preprint arXiv:1609.03499, 2016.

[9] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2016, pp. 770–778.

[10] S. Hochreiter and J. Schmidhuber, "Long short-term memory," Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997.

[11] S. Zheng, K. Ristovski, A. Farahat, and C. Gupta, "Long short-term memory network for remaining useful life estimation," in Proc. IEEE Int. Conf. Prognostics and Health Management (ICPHM), 2017, pp. 88–95.

[12] F. O. Heimes, "Recurrent neural networks for remaining useful life estimation," in Proc. Int. Conf. Prognostics and Health Management (PHM), 2008, pp. 1–6.

[13] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in Advances in Neural Information Processing Systems 30 (NeurIPS), 2017, pp. 5998–6008.

[14] Y. Mo, Q. Wu, X. Li, and B. Huang, "Remaining useful life estimation via transformer encoder enhanced by a gated convolutional unit," Journal of Intelligent Manufacturing, vol. 32, pp. 1997–2006, 2021.

[15] A. Al-Dulaimi, S. Zabihi, A. Asif, and A. Mohammadi, "A multimodal and hybrid deep neural network model for remaining useful life estimation," Computers in Industry, vol. 108, pp. 186–196, 2019.

[16] Z. Chen, M. Wu, R. Zhao, F. Guretno, R. Yan, and X. Li, "Machine remaining useful life prediction via an attention-based deep learning approach," IEEE Transactions on Industrial Electronics, vol. 68, no. 3, pp. 2521–2531, 2021.

[17] E. Ramasso and A. Saxena, "Performance benchmarking and analysis of prognostic methods for CMAPSS datasets," International Journal of Prognostics and Health Management, vol. 5, no. 2, 2014.

[18] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," Journal of Machine Learning Research, vol. 15, no. 56, pp. 1929–1958, 2014.

[19] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in Proc. Int. Conf. Learning Representations (ICLR), 2015.

[20] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in Proc. Int. Conf. Learning Representations (ICLR), 2019.

[21] L. Prechelt, "Early stopping – but when?" in Neural Networks: Tricks of the Trade. Springer, 1998, pp. 55–69.

[22] R. Pascanu, T. Mikolov, and Y. Bengio, "On the difficulty of training recurrent neural networks," in Proc. Int. Conf. Machine Learning (ICML), 2013, pp. 1310–1318.

[23] F. Wilcoxon, "Individual comparisons by ranking methods," Biometrics Bulletin, vol. 1, no. 6, pp. 80–83, 1945.

[24] K. Simonyan, A. Vedaldi, and A. Zisserman, "Deep inside convolutional networks: Visualising image classification models and saliency maps," in Proc. Int. Conf. Learning Representations (ICLR), Workshop Track, 2014.